Nakijk ERD — V2

NAKIJK V2 · Databaseproject · ERD (relationeel modelleren) · bij de Projectopgave Databases WT · 24 oktober 2026

Nagekeken: `Project Database/1_Project_ERD/Bouwcontrole ERD.drawio` (23 oktober, 17:43). Vergeleken met de versie uit Nakijk ERD V1 (22 oktober, 22:13).

> **OPMERKING**
>
> **Nieuw in deze versie: richting.** Je leerkracht keurde goed dat je ook tips en richting krijgt, zoals bij Programming Advanced. Elke bevinding in A en B heeft daarom, net als bij Programming Advanced, een blok **Richting** en een blok **Hoe je weet dat het opgelost is**. In V1 stond op die plaats *Zo controleer je het zelf*. De opmerkingen in C krijgen een kolom *Richting*. Wat niet verandert: je krijgt geen oplossing, geen SQL en geen verbeterd ERD. De afspraken staan in `docs/afspraken-databaseproject-V2.md`.
>
> Naast je ERD staat een bestand dat eindigt op `.dtmp`. draw.io maakt dat terwijl het bestand open staat. Heb je na 17:43 nog iets veranderd zonder op te slaan, dan zit dat niet in deze nakijk.
>
> Achteraan staat je ERD, met bij elke bevinding een label. Een groen vinkje staat bij wat sinds V1 opgelost of goed toegevoegd is.
>
> Deze nakijk staat in `Hulpmiddelen\Nakijk`, buiten `Project Database`. Zo komt hij nooit in de ZIP die je inlevert.

In één oogopslag

Je hebt sinds V1 drie dingen aangepakt. `Conflicten` heet nu `Conflict`, `Klanttype` heet nu `KlantType`, en `Klantcontact` heeft twee foreign keys gekregen. Die foreign keys hebben de juiste naam en een nummer, en ze staan waar ze in een associatie-entiteit horen. Daarmee staat de eerste bouwsteen van een relatie in je model.

Open blijven A1, A2 en A4. Nieuw is B3: je alternatieve sleutel tegenover je Bouwcontrole-documenten. **A1 en A2** zijn nog altijd de belangrijkste, want ze bepalen drie tellingen uit de opgave. Voor A1 heb je nu zelf een goed vergelijkingspunt getekend: je eigen `Klantcontact`. De richting bij A1 legt uit hoe je dat gebruikt.

Sinds V1

Bevinding uit V1	Nu	Status
A1. `ContactSoort` en `KlantType` bij de associatie-entiteiten	ongewijzigd	✗ open
A2. `OnderdeelType`: enumeratie of karakteristieke entiteit	ongewijzigd	✗ open
A3. `Conflicten` in het meervoud	heet nu `Conflict`	✓ opgelost
A4. `Id` en `Artikelcode` met een hoofdletter	ongewijzigd	✗ open
B1. Namen in de notitie wijken af van de tekening	`KlantType` klopt nu; `Contactsoort` nog niet	½ half
B2. `OnderdeelType` wijkt af van je Bouwcontrole-documenten	ongewijzigd	open
C3. Is de artikelcode echt uniek?	samengevoegd met een nieuwe bevinding, nu **B3**	open
C1, C2, C4 en C5	ongewijzigd. Bij C4 en C5 kwam er iets bij	open

Wat er goed zit

Wat	Waarom het goed is	Waar het staat
`Klantcontact` heeft twee foreign keys: `klantId` en `contactId`	Zo noemt de cursus een foreign key: de naam van de entiteit waarnaar ze verwijst, in camelCase, met `Id` erachter. Je schreef ze ook met een kleine letter, zoals elk attribuut.	hoofdstuk 4, *De regels*
De foreign keys staan in de associatie-entiteit	Een foreign key staat altijd aan de veel-kant van een relatie. Een associatie-entiteit is de veel-kant van allebei haar relaties. Is `Klantcontact` een associatie-entiteit, zoals je notitie zegt, dan horen haar twee foreign keys precies daar. Of `Klant` en `Contact` echt een veel-op-veelrelatie hebben, zie je pas als je de lijnen tekent.	hoofdstuk 6, *Waar staat de foreign key?* en *Veel-op-veel*
De foreign keys hebben een nummer: `FK1` en `FK2`	Elke foreign key krijgt het nummer van haar relatie. Zo kan een lezer een foreign key en een lijn aan elkaar koppelen. Lees ook C4.	hoofdstuk 4 en 6
`Conflict` staat nu in het enkelvoud	Dat was A3. Alle entiteiten volgen nu de regel uit de ERD-checklist van les 1.	hoofdstuk 4
`KlantType` schrijf je nu zoals in je notitie	Tekening en notitie zeggen hier hetzelfde. Dat was de helft van B1.	hoofdstuk 4, *Eén taal, overal*

Wat in V1 al goed zat, zit nog altijd goed: een primaire sleutel bij elke entiteit, de alternatieve sleutel bij `Onderdeel`, de vorm *List* voor een enumeratie, en je werk in een `.drawio`-bestand.

De eisen van de opgave

Eis uit de opgave	Wat er nu staat	Status
minstens 10 entiteiten	9 entiteiten, plus de lijst `OnderdeelType`	**½ 9 of 10, zie A2**
3 kernentiteiten	de lijst in je notitie is nog leeg	nog te doen
2 associatie-entiteiten	3 genoteerd. `Klantcontact` heeft nu twee foreign keys; bij de andere twee zijn geen twee entiteiten aan te wijzen	**✗ zie A1**
1 karakteristieke entiteit	`OnderdeelType` genoteerd, maar getekend als enumeratie	**✗ zie A2**
4 vrij te kiezen entiteiten	hangt af van de soorten hierboven	nog niet te beoordelen
één kernentiteit met minstens 8 attributen, de andere minstens 4	nog geen attributen	nog te doen
minstens één special topic	nog geen	nog te doen
minstens één enumeratie	`OnderdeelType`, zolang het een enumeratie is	**½ zie A2**
relaties met multipliciteit	nog geen lijnen. `FK1` en `FK2` wachten op hun relatie	nog te doen, zie C4
relatiebeschrijvingen	zeven lege plaatsen in je notitie	nog te doen, zie C4
primaire en foreign keys	een PK bij elke entiteit, twee FK's in `Klantcontact`	**½ zie A4**
datatypes en NA/NNA	nog niet	nog te doen
DTR, DTN of DTC	nog niet	nog te doen
eventuele alternatieve sleutels	bij `Onderdeel`. De ERD-checklist van les 2 vraagt er één bij elke kernentiteit	nog niet te beoordelen: de kernentiteiten zijn nog niet benoemd
benoeming van de soort	in de notitie, voor 3 van de 9 entiteiten en voor `OnderdeelType`	**½ zie A1, A2 en B1**

A. Wat tegen een regel of eis ingaat

A1. `ContactSoort` en `KlantType` staan nog bij de associatie-entiteiten

Waar: de notitie rechtsonder, onder *associatie-entiteiten*, en de entiteiten `Contactsoort` en `KlantType`.

Wat. Een associatie-entiteit stelt een veel-op-veelrelatie voor tussen twee entiteiten, en heeft een foreign key naar elk van beide. Bij `Klantcontact` staat dat nu zelfs in je model: `FK1` verwijst naar `Klant`, `FK2` naar `Contact`. Bij `ContactSoort` en `KlantType` kan je die twee entiteiten in je model niet aanwijzen: ze hebben geen foreign keys, er lopen geen lijnen naartoe, en hun naam verwijst niet naar twee entiteiten uit je model. De naam alleen beslist het niet: hoofdstuk 8 zegt dat een associatie-entiteit soms een eigen naam heeft, zoals `Beoordeling`. Het gaat om wat de entiteit betekent. De richting hieronder helpt je dat na te gaan.

Waarom het telt.

- De opgave vraagt de benoeming van de soort bij elke entiteit (*Vereisten*), en heeft een eigen feedbackcriterium *Benoemen entiteiten*.
- Kan je bij deze twee geen twee entiteiten aanwijzen, dan blijft er één associatie-entiteit over. De opgave vraagt er twee.
- De telling van de andere soorten verandert mee.

Richting.

- Je hebt het patroon nu zelf getekend in `Klantcontact`. Leg `ContactSoort` en `KlantType` ernaast, en probeer voor elk de twee foreign keys op te schrijven zoals je bij `Klantcontact` deed. Naar welke twee entiteiten uit je model zouden ze verwijzen? Lukt dat niet, dan weet je dat het geen associatie-entiteit is.
- Ga in dat geval verder in de tabel *Hoe je de soort herkent* (hoofdstuk 8). De eerste vraag heb je dan beantwoord. Stel de volgende vragen in volgorde.
- Het voorbeeld uit de les in hoofdstuk 8, *Hoe het eruitziet*, gaat over een klant en hoe je hem bereikt: `Klant`, `Communicatie` en `KlantCommunicatie`. Dat lijkt op je hoek rechtsboven. Kijk welke soort elk van die drie heeft, en vooral waarom. Stel je daarna dezelfde vraag bij je eigen entiteiten.
- Pas daarna je notitie aan, en kijk of de tellingen in de eisentabel hierboven mee veranderen.

Hoe je weet dat het opgelost is. Elke entiteit onder *associatie-entiteiten* heeft, net als `Klantcontact`, twee foreign keys naar twee entiteiten uit je model. Je kan bij elk de zin in beide richtingen schrijven: *een … kan bij meerdere … horen, en een … kan bij meerdere … horen*. Wat daar niet aan voldoet, staat bij een andere soort, met de reden uit de tabel van hoofdstuk 8.

A2. `OnderdeelType` is getekend als enumeratie, maar benoemd als karakteristieke entiteit

Waar: de lijst `OnderdeelType`, midden onderaan, en de notitie onder *karakteristieke entiteit*.

Wat. De tekening zegt **enumeratie**: de vorm *List*, met de waarden eronder en zonder primaire sleutel. De notitie zegt **karakteristieke entiteit**: een entiteit op zich, met een eigen primaire sleutel. Allebei tegelijk kan niet, en wie je model beoordeelt, kan niet weten welke van de twee je bedoelt.

Waarom het telt.

- In de verdeling van de entiteiten vraagt de opgave één karakteristieke entiteit. Daarnaast vraagt ze, apart, minstens één enumeratie. Eén element kan niet voor allebei tellen.
- Een enumeratie is geen entiteit. Of je model nu 9 of 10 entiteiten telt, hangt ervan af wat `OnderdeelType` is.

Richting.

- Beslis eerst wat `OnderdeelType` moet zijn. Maak pas daarna je tekening en je notitie gelijk. Zo pas je maar één keer aan.
- Gebruik de vuistregel uit les 3, in hoofdstuk 9, *Hoe je kiest*. Stel de twee vragen voor jouw lijst. Is ze klein en stabiel, of komen er geregeld types bij? Wil je aan een onderdeeltype eigen gegevens of relaties hangen?
- Kijk breder dan deze ene lijst. De opgave vraagt een karakteristieke entiteit én een enumeratie. Wat je hier kiest, bepaalt dus wat je elders in je model nog nodig hebt.
- Doe eerst taak L04 van je takenlijst: welk special topic uit les 12 bevat je bron al? Het antwoord kan meebepalen wat `OnderdeelType` moet zijn.
- **Valkuil.** Een lijst die door niets gebruikt wordt, zegt niets over je onderdelen. Hoofdstuk 9 toont hoe elk van de twee vormen in een model gebruikt wordt. Een enumeratie heeft geen relatie; een karakteristieke entiteit wel.

Hoe je weet dat het opgelost is. Tekening en notitie zeggen hetzelfde. Is het een enumeratie, dan gebruikt een attribuut in een entiteit haar. Het leerboek noteert dat als datatype, maar de dia's leggen die notatie niet vast: vraag na hoe je leerkracht ze wil (hoofdstuk 9). Is het een karakteristieke entiteit, dan heeft ze een primaire sleutel, en loopt er een relatie naar de entiteit die ze beschrijft, met de foreign key aan de veel-kant. En de tellingen in de eisentabel kloppen.

A4. De primaire sleutel en de artikelcode beginnen nog met een hoofdletter

Waar: de eerste rij van elke entiteit (`Id`, negen keer), en de tweede rij van `Onderdeel` (`Artikelcode`).

Wat. In de cursus begint elk attribuut met een kleine letter, ook de primaire sleutel. Een naam van meer woorden schrijf je in camelCase.

Waarom het telt. De ERD-checklist van les 1 zegt: *elk attribuut begint met een kleine letter*.

Richting.

- In `Klantcontact` deed je het al goed: `klantId` en `contactId` beginnen met een kleine letter. Dezelfde afspraak geldt voor elk ander attribuut.
- Kijk in hoofdstuk 4, *De regels*, naar de rij *primaire sleutel*. Daar staat de naam die de cursus altijd gebruikt.
- **Valkuil.** In C# is het net omgekeerd: daar begint een eigenschap met een hoofdletter, zoals `Id` en `KrakenId` in je klasse `Onderdeel`. Dat is daar juist. Pas je C#-klassen dus niet aan je ERD aan. Twee vakken, twee afspraken.

Hoe je weet dat het opgelost is. Geen enkel attribuut in je model begint nog met een hoofdletter. Loop elke entiteit van boven naar onder af. Met Ctrl+F in draw.io vind je snel waar `Id` nog staat.

B. Wat elkaar tegenspreekt

B1. `Contactsoort` heet in de notitie anders dan in de tekening

Waar: de entiteit rechts, en de notitie onder *associatie-entiteiten*.

Wat. `KlantType` staat nu in tekening en notitie hetzelfde. `Contactsoort` nog niet: in de notitie staat `ContactSoort`, met een hoofdletter in het midden. In de tekening schrijf je `KlantType` en `OnderdeelType` met een hoofdletter in het midden, en `Contactsoort` zonder.

Waarom het telt. Naamgeving moet consequent gebeuren (hoofdstuk 4, *Eén taal, overal*). Wie je model beoordeelt, legt je notitie naast je tekening. Twee schrijfwijzen lezen als twee verschillende dingen.

Richting.

- Kies één manier om een samengestelde entiteitsnaam te schrijven, en hou je eraan in de tekening én in de notitie. De voorbeelden in de cursus schrijven elk deel met een hoofdletter, zoals `KlantCommunicatie` en `ReceptIngredient` (hoofdstuk 4 en 8). Een andere keuze kan ook, zolang ze overal dezelfde is.
- **Valkuil.** Ook `Klantcontact` is een samengestelde naam. Neem hem mee als je je keuze over het hele model doorvoert.
- **Tip.** De soort staat nu in een aparte notitie. In de voorbeeldmodellen van de cursus staat ze boven de naam van elke entiteit (hoofdstuk 8), en je eigen taak E10 zegt hetzelfde. Een aparte lijst moet je bij elke wijziging mee aanpassen. Dat ze nu al afwijkt, toont dat risico.

Hoe je weet dat het opgelost is. Elke naam staat letter per letter hetzelfde in tekening en notitie, en al je samengestelde namen volgen dezelfde schrijfwijze.

B2. Je lijst `OnderdeelType` wijkt af van je eigen Bouwcontrole-documenten

Waar: de waarden van `OnderdeelType`.

Wat. Ongewijzigd sinds V1. Je C#-modellen in Bouwcontrole en je document *Eigenschappen per onderdeeltype* tellen elk tien onderdeeltypes. Je lijst telt er ook tien, maar niet dezelfde tien:

Wat	In je C#-modellen	In je eigenschappen-document	In je lijst
een koeler	`ProcessorKoeler`	koeler	staat er niet in
een besturingssysteem	staat er niet in	staat er niet in	besturingssysteem

De andere negen zitten in alle drie, soms onder een andere naam, zoals processor en CPU.

Waarom het telt. De opgave vraagt een ERD dat de bron zo getrouw mogelijk nabouwt. Je bron is de werkvloer van Alternate, en je Bouwcontrole-documenten beschrijven die. Een verschil kan bewust zijn, of niet. Aandachtspunt A04 van je takenlijst gaat hierover: de twee maatstaven.

Richting.

- Beslis eerst welke maatstaf voorgaat: je bron getrouw nabouwen, zoals de opgave vraagt, of enkel wat Bouwcontrole nodig heeft (aandachtspunt A04). Pas die maatstaf daarna toe op elk verschil. Een hulpvraag: wordt het in een pc gebouwd, of met de order meegeleverd?
- Wat je bewust weglaat of toevoegt, schrijf je in een README of Word-document bij je ERD. De opgave raadt dat aan voor elke keuze die verwarring kan geven (*Verduidelijkingen*). Begin er nu al mee, want achteraf weet je vaak niet meer waarom.
- Taak L12 van je takenlijst is daarvoor gemaakt: daar hou je bij waar je ERD afwijkt van je attributenlijst, met wat je toevoegde en wat je wegliet.

Hoe je weet dat het opgelost is. Elk verschil tussen je lijst en je Bouwcontrole-modellen is weg, of staat uitgelegd in je verantwoording.

B3. Je alternatieve sleutel is de artikelcode, je Bouwcontrole-documenten gebruiken het Kraken-Id

Waar: de tweede rij van `Onderdeel`.

Wat. In je model is `Artikelcode` de alternatieve sleutel van `Onderdeel`. In je Bouwcontrole-documenten is het **Kraken-Id** de unieke code waarop gescand en gezocht wordt. De artikelcode staat daar bij de open vragen: ze zit in geen enkel C#-model, en het is nog niet beslist of ze naast het Kraken-Id blijft bestaan (je hoofdmodel-document en *Hoofdmodel Onderdeel V1*). Dit vervangt C3 uit V1.

Waarom het telt. Een alternatieve sleutel moet echt uniek zijn (hoofdstuk 5, *Veelgemaakte fouten*). En de opgave vraagt dat je de bron zo getrouw mogelijk nabouwt. Bestaan er in je bron twee codes, dan is het een keuze welke je opneemt, en waarom.

Richting.

- Ga na welke codes er bij Alternate echt bestaan voor één product, en welke daarvan nooit twee keer voorkomt. Vraag het na op je werk als je twijfelt.
- Lees in hoofdstuk 5 het verschil tussen een kandidaatsleutel en een alternatieve sleutel, en welke eigenschappen een goede sleutel heeft.
- Hou het verschil met Bouwcontrole bij in taak L12, en verantwoord je keuze in je README.

Hoe je weet dat het opgelost is. Je kan voor elke alternatieve sleutel in `Onderdeel` zeggen waarom ze uniek is, en het verschil met je Bouwcontrole-documenten is weg of uitgelegd.

C. Opmerkingen

Geen fouten, wel goed om nu al te weten:

Nr.	Waar	Wat en waarom	Richting
C1	`Order`	ORDER is een gereserveerd woord in SQL Server, zoals in ORDER BY. Voor het ERD is het geen fout: de cursus gebruikt zelf `Order`, in het voorbeeld van Harry en Saartje. Maar SQL Server aanvaardt een gereserveerd woord enkel als naam met een speciale schrijfwijze.	Voor het ERD hoef je niets te doen. Beslis voor je aan de SQL begint of de naam blijft. Blijft hij, zoek dan in de documentatie van SQL Server op *delimited identifiers*.
C2	de waarden van `OnderdeelType`	Ze zijn niet op één manier geschreven: `CPU`, `SSD` en `HDD` in hoofdletters, de andere in kleine letters, en `grafische kaart` met een spatie. De cursus wil dat je consequent bent (hoofdstuk 4), en een vaste lijst bestaat net om een waarde overal hetzelfde te schrijven (hoofdstuk 9).	Kies één schrijfwijze voor de waarden en pas ze overal toe. Bedenk dat je die waarden in het SQL-deel letterlijk zult typen.
C4	de notitie, *Relatiebeschrijving*, en `Klantcontact`	Je notitie heeft zeven plaatsen die allemaal `R1` heten. **Nieuw:** `FK1` en `FK2` hebben nog geen relatie. Een foreign key hoort bij de relatie met hetzelfde nummer: `FK1` bij `R1`, `FK2` bij `R2` (hoofdstuk 4 en 6).	Teken de twee relaties waar je foreign keys bij horen, en geef elke lijn het nummer van haar foreign key. Nummer daarna je relatiebeschrijvingen in de notitie mee.
C5	de map `1_Project_ERD`	Naast je ERD staan `.$Bouwcontrole ERD.drawio.bkp` en, **nieuw**, `.$Bouwcontrole ERD.drawio.dtmp`. draw.io maakt die bestanden zelf aan. Ze horen niet bij wat je inlevert: de opgave vraagt het `.drawio`-bestand.	Lever enkel `Bouwcontrole ERD.drawio` in. Wil je ze niet op GitHub, sluit ze dan uit met een `.gitignore`, zoals taak V13 van je takenlijst dat voorstelt voor de tijdelijke Word-bestanden.

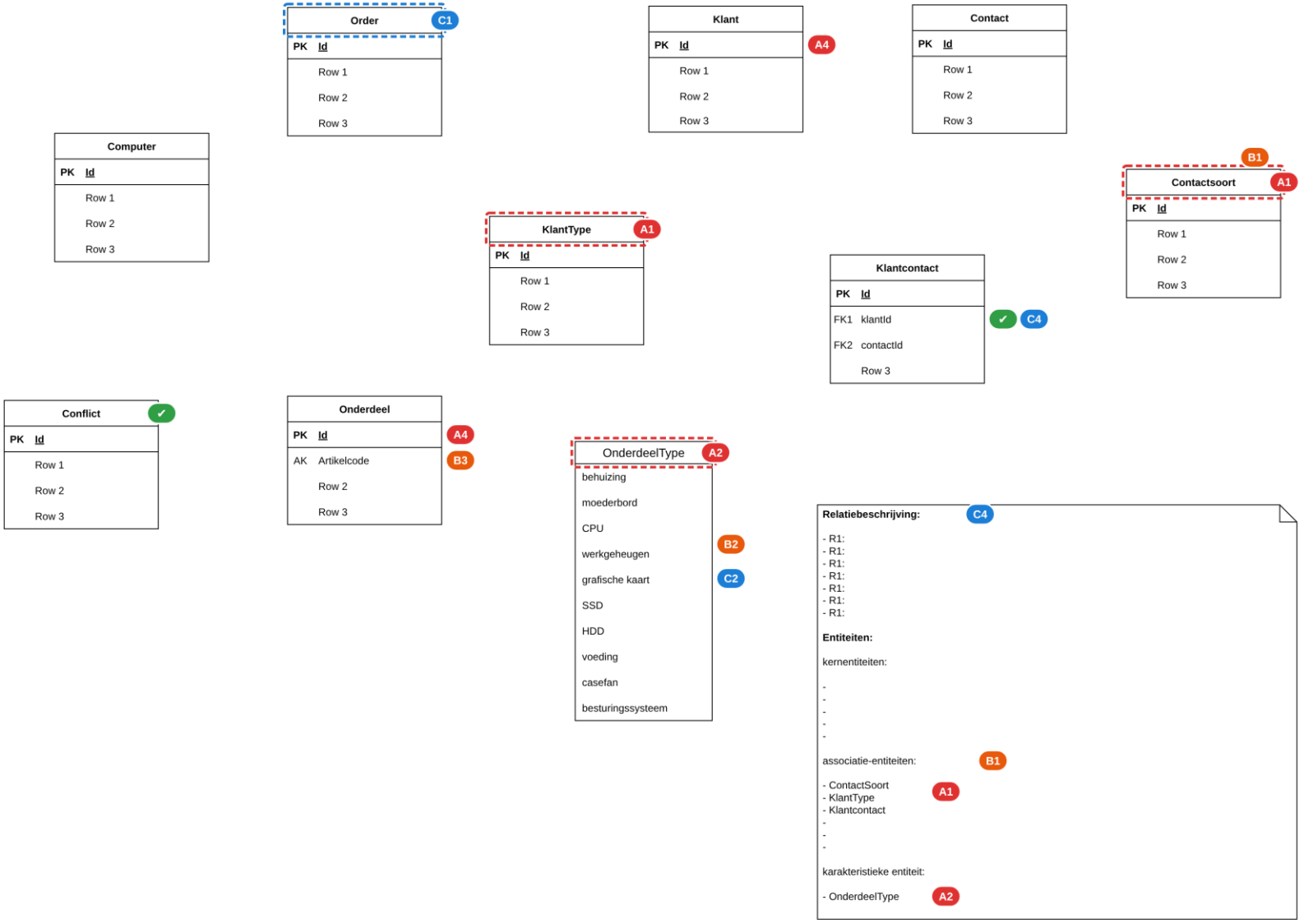
Wat ik nog niet kon beoordelen

Dit staat nog niet in je model. Ik kijk het na in een volgende versie, en dan lopen ook de zeven feedbackcriteria van de opgave volledig mee.

- de relaties: de lijnen, de veel-op-veelrelaties, de multipliciteiten en de relatiebeschrijvingen. Bij `Klantcontact` staan de foreign keys al klaar.
- de attributen en hun aantallen, de datatypes, NA en NNA
- de delete-opties
- het special topic
- wat één rij van `Contact`, `Contactsoort`, `Computer` en `Conflict` voorstelt. Zonder attributen of relaties kan een lezer dat nog niet afleiden.

Richting voor de volgende stap. Het conceptuele model (hoofdstuk 2) bestaat uit de kernentiteiten en de lijnen ertussen, nog zonder attributen. Je hebt bij `Klantcontact` al getoond dat je weet waar een foreign key hoort. Teken de lijnen tussen je entiteiten, en stel aan beide kanten van elke lijn de twee vragen uit hoofdstuk 7: *kan het ene bestaan zonder het andere?* en *hoeveel van het ene horen bij één van het andere?* Komt er aan beide kanten *veel* uit, dan heb je een veel-op-veelrelatie gevonden. Hoofdstuk 6 legt uit wat je daarmee doet.

Bijlage: je ERD, met de bevindingen erop



Je ERD zoals het nu is, nagetekend uit je .drawio-bestand. Er is niets aan veranderd. De labels tonen waar elke bevinding zit: rood is A (tegen een regel), oranje is B (tegenstrijdig), blauw is C (opmerking). Een groen vinkje staat bij wat sinds V1 opgelost of goed toegevoegd is.